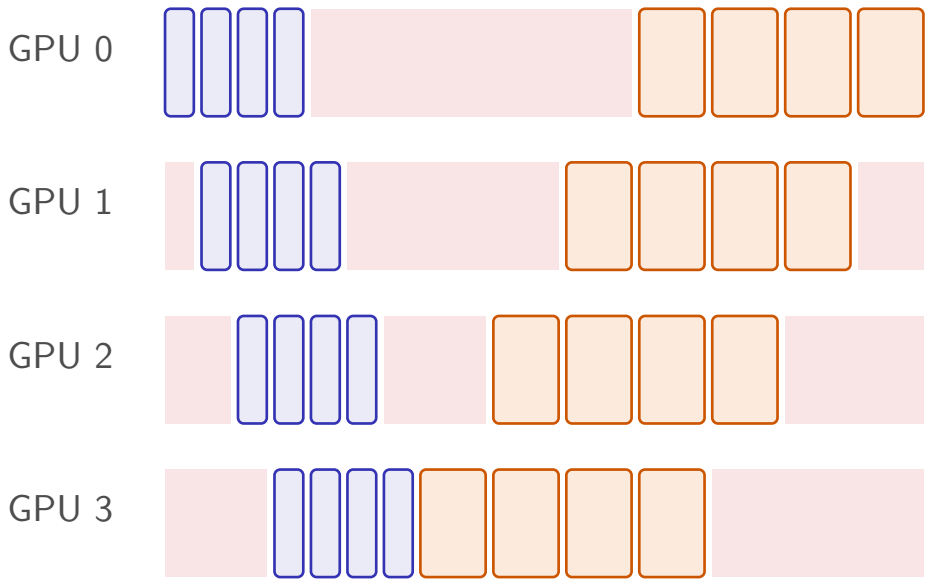
Pipeline and 3D Parallelism

Cutting a model into stages, filling the pipeline, and choosing a d $\times$ t $\times$ p layout

Yuke Wang

Rice University

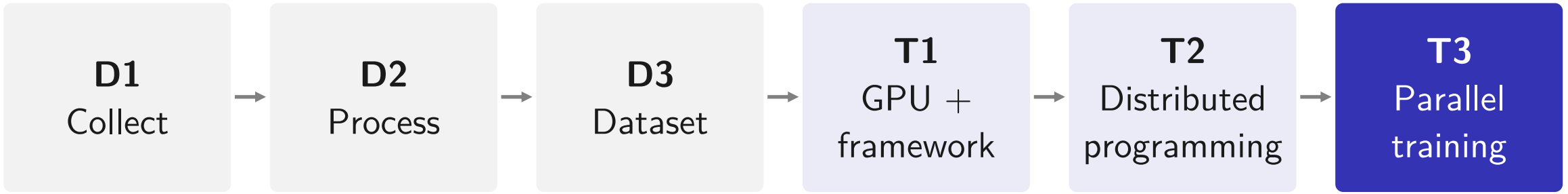
COMP 329 · Spring 2027 · Week 13, Lecture 1



GPipe schedule: 4 stages, 4 microbatches

blue: forward · orange: backward · red: idle

Where we are in the pipeline



Week 12: data parallelism, ZeRO / FSDP; tensor parallelism

In our pipeline

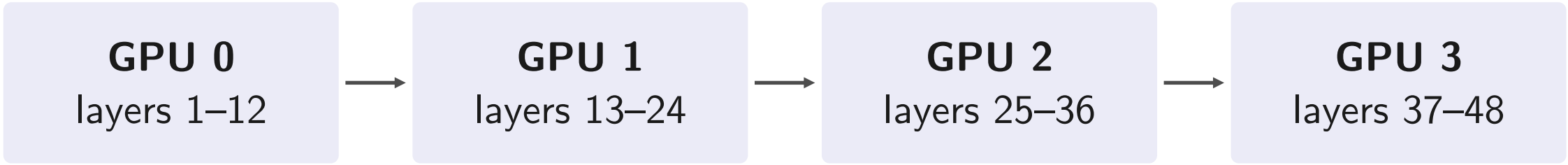
Today: split the model **by layers** across GPUs, then combine data $\times$ tensor $\times$ pipeline into one layout.

After this lecture: **T3 checkpoint** (DP update, basic layout tests) and **Lab 5** are due.

Learning goals for today

- Partition a model into **pipeline stages** and size the messages between them.
- Derive the **bubble fraction** $(p-1)/(m+p-1)$ from a schedule timeline.
- Compare **GPipe** and **1F1B** schedules on time and activation memory.
- Map a $\mathbf{d} \times \mathbf{t} \times \mathbf{p}$ layout onto servers and estimate memory and communication per GPU.
- Choose a layout for a fixed GPU count and memory budget, **with numbers**.

A puzzle to start



A 48-layer model is split across 4 GPUs, 12 layers each. One batch goes through forward, then backward. What fraction of the time is each GPU busy?

Key idea

Only **25%**: layer 13 must wait for layer 12, so one GPU works at a time. Split the batch into 8 microbatches and it rises to $\mathbf{8/11 \approx 73\%}$.

Part 1

Cutting the model into stages

What if the model does not fit, and splitting every layer is too chatty?

Why a third kind of parallelism?

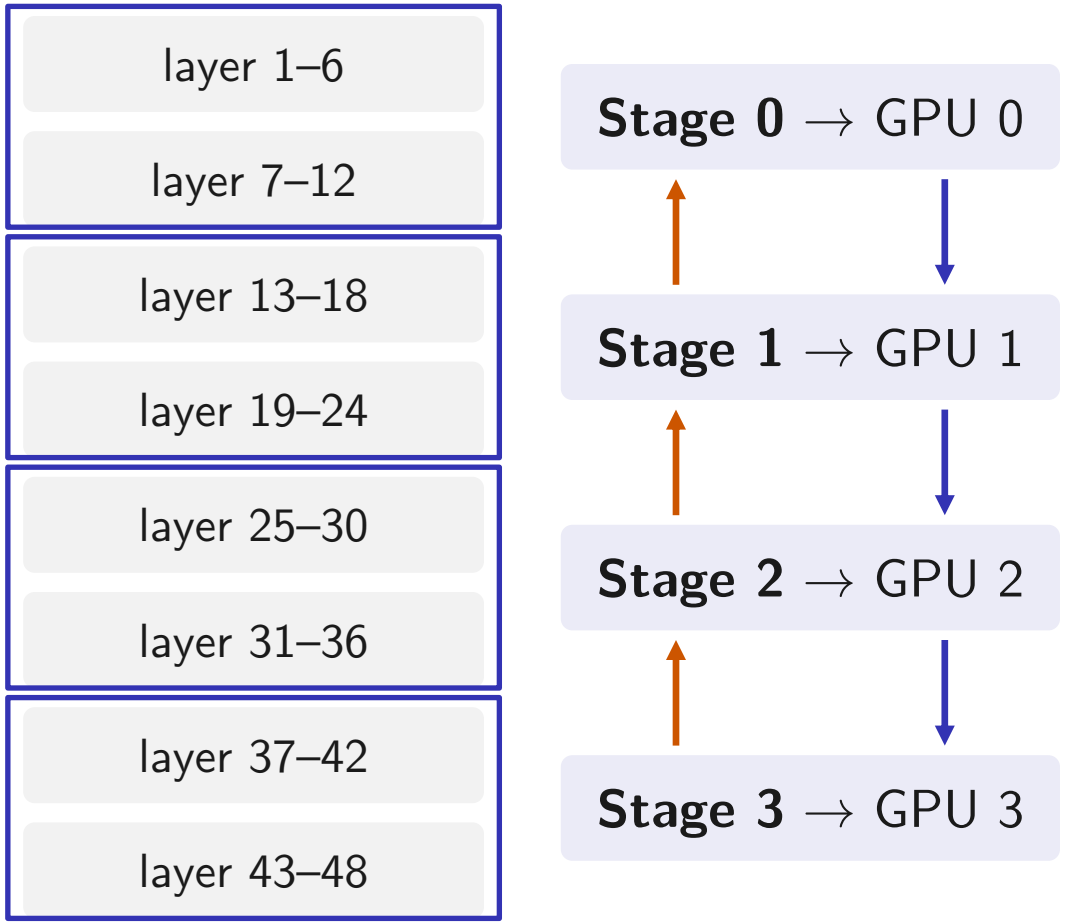
	Splits	Talks	Limit
Data (d)	the batch	gradients, once per step	model must fit (or shard)
Tensor (t)	each weight matrix	4 all-reduces per layer	fast links: one server
Pipeline (p)	the layers	activations at each cut	idle time (bubbles)

Key idea

Pipeline parallelism stores **1/p of the layers** per GPU and sends only the tensors that cross a cut.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

Layer stages



Forward: send activation h at each cut to the next GPU

Backward: send the gradient of the loss w.r.t. h back

Key idea

Each GPU holds **L/p layers** and their optimizer state.

What crosses a stage boundary

$$\text{message} = b \cdot s \cdot h \text{ elements}$$

$$= 1 \cdot 2048 \cdot 8192 \times 2 \text{ bytes} \approx 33.6 \text{ MB}$$

$$\times\ 2 \text{ sends per microbatch}$$

Per microbatch	**Size**
Pipeline: one cut	2 bsh
Tensor: 24 layers, t = 8	168 bsh

Memory

Pipeline traffic is $\approx$ **80× smaller** than tensor-parallel traffic: it can cross the slower links **between** servers.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

Point-to-point messages in code

```
# stage s sends forward ...
h = stage(x)
dist.send(h.detach(), dst=rank + 1)

# ... stage s + 1 receives
h = torch.empty(b, 64)            # (b, s, h) for LLMs
dist.recv(h, src=rank - 1)
h.requires_grad_()

# backward: gradient flows the other way
dist.send(h.grad, dst=rank - 1)
```

- `send`/`recv` block; `isend`/`irecv` return a request to `wait()`.
- The receiver allocates the buffer: **shape and dtype** must match.
- `detach()` on send; `requires_grad_()` on receive.

Source: PyTorch documentation, torch.distributed: send, recv, isend, irecv.

Balancing the stages: our deeper toy model

Layer	Params	Fwd FLOPs
Embedding 322×16	5,152	≈ 0 (lookup)
Hidden 1: $128 \to 64$	8,256	16,384
Hidden 2–4: $64 \to 64$ (each)	4,160	8,192
Output: $64 \to 322$	20,930	41,216
Total	**46,818**	**82,176**

forward FLOPs per example

Cut by layer count

stage 0: 24,576 · stage 1: 57,600

$\to$ stage 1 is 2.3$\times$ slower

Cut by FLOPs

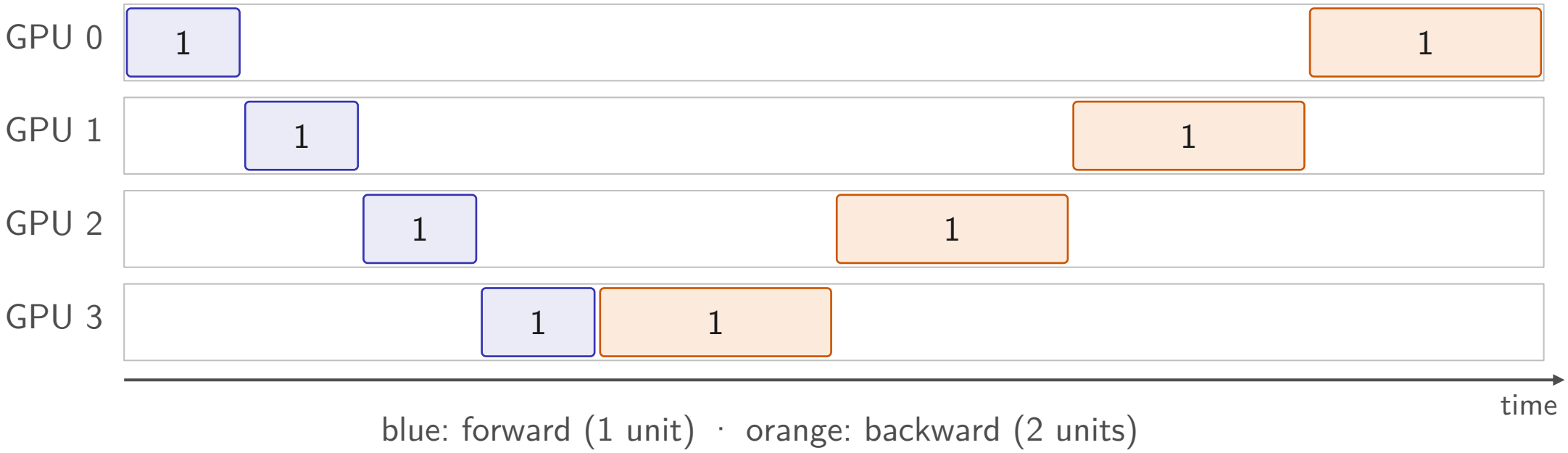
stage 0: embed + hidden 1–4

$= 40{,}960$ · stage 1: 41,216

Watch out

A pipeline runs at the pace of its **slowest stage**: balance FLOPs, not layer counts.

The naive schedule, one step at a time



Watch out

At every moment **exactly one** GPU is busy. This is just a slower single GPU with more memory.

Naive pipeline: utilization

$$\text{step time} = p \cdot (\text{tF} + \text{tB})$$

$$\text{busy time per GPU} = \text{tF} + \text{tB}$$

$$\text{utilization} = 1 \,/\, p, \quad \text{idle} = (p-1) \,/\, p$$

p	2	4	8
busy	50%	25%	12.5%

Watch out

Naive pipelining fixes **memory**, not **speed**.

Check yourself

Two minutes with a neighbour.

- b $= 4$, s $= 1024$, h $= 4096$ in BF16: how many bytes cross one cut per microbatch?
- Can our deeper MLP be balanced across **3** stages? Why or why not?
- Naive pipeline over 8 GPUs: what fraction of the time is each GPU busy?

Part 2

Microbatches and the bubble

How do we keep every stage busy?

GPipe: split the batch into microbatches

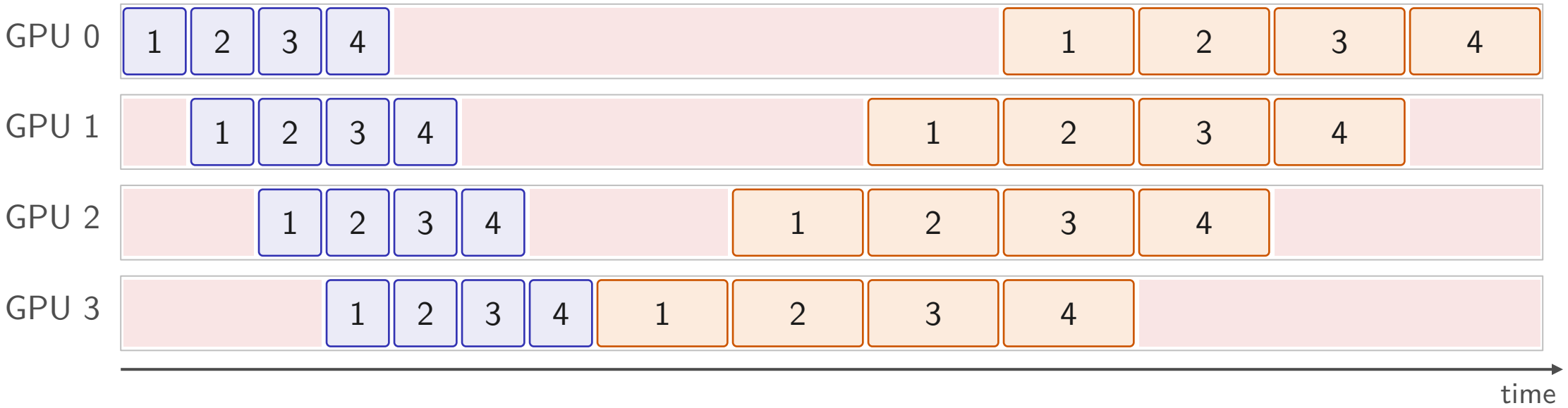
$$\ell(B) = (1/B)\ \Sigma\ \ell(\text{example}) = (1/m)\ \Sigma\ \ell(\text{microbatch } j)$$

$$\nabla\ell(B) = (1/m)\ \Sigma\ \nabla\ell(\text{microbatch } j)$$

- Run m microbatches through the stages like an **assembly line**.
- **Accumulate** gradients; one optimizer step after the last one (a **flush**).
- Same update as the full batch: *synchronous* pipelining.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

The GPipe schedule: p = 4, m = 4



red: bubble (idle)

Key idea

Each GPU is busy **12 of 21** units: the bubble is $\mathbf{9/21 \approx 43\%}$.

Deriving the bubble fraction

$$\text{step time} = (m + p - 1) \cdot (\text{tF} + \text{tB})$$

$$\text{busy time} = m \cdot (\text{tF} + \text{tB})$$

$$\text{bubble fraction} = (p - 1) \;/\; (m + p - 1)$$

$$\text{bubble} \div \text{ideal time} = (p - 1) \;/\; m \quad \text{(Megatron-LM form)}$$

Key idea

m $= 1$ gives the naive (p $-$ 1)/p. As m $\to \infty$ the bubble vanishes: we need **m** $\gg$ **p**.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

How many microbatches are enough?

	$\mathbf{m = 1}$	$\mathbf{m = 4}$	$\mathbf{m = 8}$	$\mathbf{m = 16}$	$\mathbf{m = 32}$
$p = 2$	50%	20%	11%	5.9%	3.0%
$p = 4$	75%	43%	27%	16%	8.6%
$p = 8$	88%	64%	47%	30%	18%

bubble fraction $(p - 1)/(m + p - 1)$

Key idea

GPipe's rule of thumb: $\mathbf{m \geq 4p}$ microbatches. Deeper pipelines need more microbatches.

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

Microbatches are not free

$$B = d \cdot m \cdot b \quad \text{(fixed by the optimizer)}$$

- Smaller b $\rightarrow$ **smaller matmuls** $\rightarrow$ lower FLOP/s per GPU (Week 8).
- More messages $\rightarrow$ each pays the **latency** α (Week 11, Lab 4).
- Fixed **per-microbatch overhead**: launches, Python — dominant for our MLP.

Watch out

Pick m large enough to shrink the bubble, b large enough to keep kernels efficient.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. Takeaway #3.

GPipe's hidden cost: activation memory

$$\text{stored at the peak} \;\propto\; m \;\text{ microbatches}$$

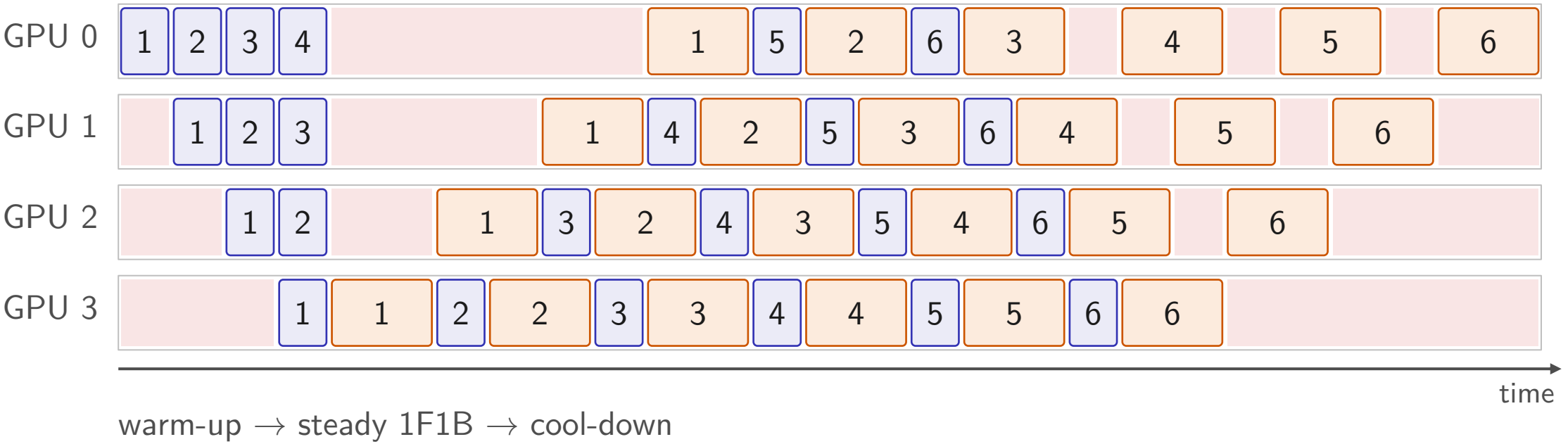
- Backward of microbatch j needs **its** saved activations.
- GPipe runs **all m forwards first**: every stage holds m sets at once.
- Small bubble needs large m $\rightarrow$ large memory: the two goals **conflict**.

Watch out

Fixes: **recompute** activations in backward (GPipe), or **reorder** the schedule (1F1B).

Source: Huang et al., "GPipe," NeurIPS 2019; Narayanan et al., SC 2021, §2.2.

1F1B (PipeDream-Flush): p = 4, m = 6



Key idea

Same step time, same bubble **3/9**; but each microbatch's activations are freed **early**.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §2.2.1.

1F1B bounds the activations in flight

Schedule	Bubble	In flight (stage 0)	m = 6	m = 32
GPipe	$(p-1)/(m+p-1)$	m	6	32
1F1B	$(p-1)/(m+p-1)$	p	4	4

p = 4; counts from the schedule simulator

$$\text{stage 0 stores } \; p \; \text{ microbatches} \times L/\text{p layers} \; = \; L \text{ layers' worth}$$

Key idea

1F1B lets us raise **m** (smaller bubble) **without** raising activation memory.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §2.2.1.

Interleaved schedules (for reference)

GPU 0	layers 1–6	layers 25–30
GPU 1	layers 7–12	layers 31–36
GPU 2	layers 13–18	layers 37–42
GPU 3	layers 19–24	layers 43–48

p = 4, v = 2 chunks per GPU (48 layers)

$$\text{bubble} \div \text{ideal} = (p-1)/(v \cdot m)$$

- Bubble shrinks by **v**.
- Point-to-point traffic grows by **v**.
- m must be a multiple of p.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §2.2.2.

Check yourself

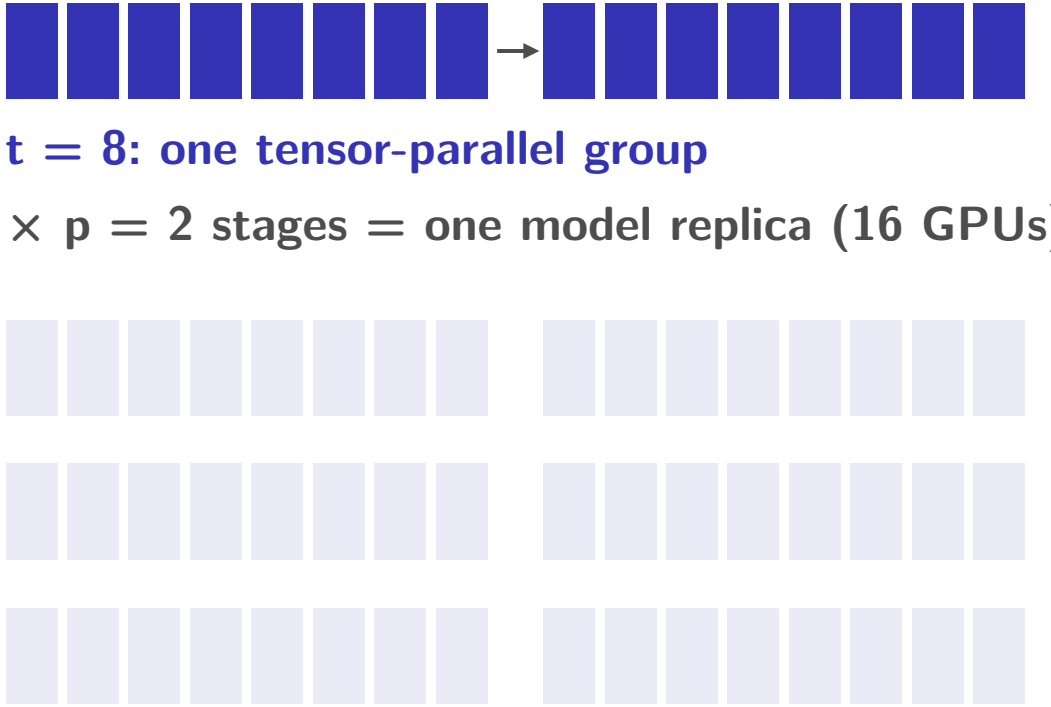
- $p = 8$, $m = 16$: what is the bubble fraction? What m brings it under 10%?
- Same p and m: how many microbatches' activations does GPipe store? 1F1B?
- Global batch 512 sequences, $d = 8$, $m = 16$: what is the microbatch size b?

Part 3

Composing 3D parallelism

Given 64 GPUs, how many go to each kind of parallelism?

World size = d × t × p



t = 8: one tensor-parallel group

× p = 2 stages = one model replica (16 GPUs)

× d = 4 replicas = 64 GPUs

$$N = d \cdot t \cdot p$$

$$64 = 4 \cdot 8 \cdot 2$$

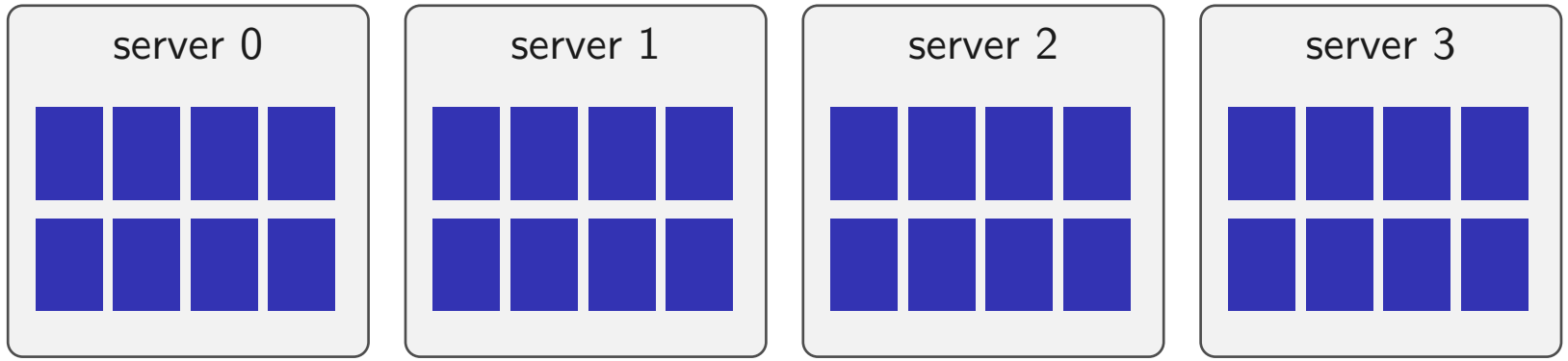
$$\text{rank} = i + t \cdot (j + p \cdot k)$$

i: tensor slice · j: stage · k: replica

Key idea

Each rank belongs to **one** group of each kind.

Mapping the layout onto servers



... 8 servers

NVLink: 450 GB/s per direction $\rightarrow$ **t** (every layer)

InfiniBand: 50 GB/s per GPU per direction $\rightarrow$ **p** (small sends)

... and **d** outermost: one gradient all-reduce per step, overlapped

Key idea

Chattiest dimension on the **fastest** link.

Source: H100 datasheet (NVLink 900 GB/s bidirectional); NVIDIA DGX H100 datasheet (8 $\times$ 400 Gb/s ConnectX-7 for the compute fabric).

Memory per GPU: parameters and optimizer

Per parameter	Bytes
Weights (BF16)	2
Gradients (BF16)	2
Master weights (FP32)	4
Adam m and v (FP32)	$4 + 4$
Total	**16**

$$\text{memory} \approx 16\ \Psi\ /\ (t \cdot p)$$

$t \cdot p$	1	8	16	32
GB / GPU	640	80	40	20

$\Psi = 40\text{B}$

Memory

With 80 GB GPUs, a 40B model needs $\mathbf{t \cdot p \geq 16}$ before counting activations.

Source: Rajbhandari et al., "ZeRO," SC 2020 (16 bytes per parameter for mixed-precision Adam).

Memory per GPU: activations

$$\text{per layer} \approx 34 \cdot s \cdot b \cdot h \text{ bytes} \;/\; t$$

$$\text{stage 0 holds } \; p \cdot (L \;/\; p) = L \; \text{ layers' worth}$$

$$48 \times 34 \times 2048 \times 8192 \;/\; 8 \approx 3.4 \text{ GB}$$

Memory

Activations shrink with **t** (and b), not with p. With t = 8: 80 + 3.4 GB **does not fit**; 40 + 3.4 GB does.

Source: Korthikanti et al., "Reducing Activation Recomputation in Large Transformer Models," MLSys 2023.

Communication per step, per dimension

Dimension	Bytes per GPU per step	(4, 8, 2)	Time
Tensor (NVLink)	$8bsh \cdot (t-1)/t \cdot 2 \cdot (L/p) \cdot m$	90 GB	0.20 s
Pipeline (IB)	$2 \cdot bsh \cdot 2 \cdot m$	1.1 GB	0.02 s
Data (IB)	$2(d-1)/d \cdot 2\Psi/(t \cdot p)$	7.5 GB	0.15 s

Key idea

Tensor traffic is the **largest**: keep it on NVLink. Data traffic is the largest on the **slow** link, but happens once per step.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §3.2; link speeds as on the previous slides.

Worked layout: 64 GPUs, a 40B model

- **Memory**: $16\Psi/(t \cdot p) + 3.4\ \text{GB} \leq 80\ \text{GB} \Rightarrow t \cdot p \geq 16$
- **Links**: tensor traffic on NVLink $\Rightarrow$ $t \leq 8$, take $\mathbf{t = 8}$
- **Bubble**: smallest pipeline that fits $\Rightarrow$ $\mathbf{p = 2}$, $m = 16$
- **Rest** is data parallel: $d = 64 \,/\, 16 = \mathbf{4}$

Result

Layout $\mathbf{(d, t, p) = (4, 8, 2)}$: 43 GB per GPU, tensor groups inside servers, pipeline across.

Worked layout: predicted step time

$$\text{compute} = 6\Psi \cdot \text{tokens} \;/\; (N \cdot \pi \cdot 0.5) \approx 0.99$$

$$\text{tensor} \approx 0.20, \quad \text{pipeline} \approx 0.02, \quad \text{data} \approx 0.15$$

$$T \approx (\text{compute} + \text{tensor}) \cdot (1 + (p-1)/m) + \text{pipeline} + \text{data}$$

$$T \approx (0.99 + 0.20) \cdot (1 + 1/16) + 0.02 + 0.15 \approx 1.44$$

all times in seconds · tokens $= 64 \times 2048$ · $\pi = 989$ TFLOP/s (BF16) · $0.5 =$ assumed efficiency

Key idea

Compute dominates; tensor traffic is next. The bubble costs $\approx$ **6%** at m $= 16$.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). 50% of BF16 peak is an assumption.

Rules of thumb from Megatron-LM

Takeaway #1

Use tensor parallelism up to the GPUs in **one server**; use pipeline parallelism across servers.

Takeaway #2

Make t · p just large enough for the model to **fit**; give the remaining GPUs to **data** parallelism.

Takeaway #3

The best microbatch size depends on the model, p, d, and the global batch: **measure**.

Source: Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021. §3 (paraphrased).

Check yourself

- $\Psi = 13$B on 32 GPUs (80 GB): is t $\cdot$ p $= 8$ enough for parameters + Adam?
- Why is t $= 16$ a bad idea on 8-GPU servers? Which term grows, and by how much?
- (d, t, p) $= (4, 8, 2)$, rank $=$ i $+$ t $\cdot$ (j $+$ p $\cdot$ k): where does rank 37 sit?

IN-CLASS BUILD

Bubbles and 3D costs

Work in pairs. Predict first; measure in Lab 5.

Exercise 1: a two-stage pipeline

m (microbatches)	1	2	4	8
microbatch b = 64 / m	64	32	16	8
predicted bubble				
measured bubble (Lab 5)				

SETUP

Deeper MLP; stage 0 = embedding + hidden 1–4, stage 1 = output + loss; batch 64; GPipe schedule.

STRETCH

Message size per microbatch at m = 4? Is it bandwidth- or latency-bound?

Starter: stage 1 of a GPipe step

```
saved = []
for y in ys:                        # m forwards
    h = torch.empty(b, 64)
    dist.recv(h, src=0)
    h.requires_grad_()
    out = stage1(h)
    loss = F.cross_entropy(out, y) / m
    saved.append((h, loss))
for h, loss in saved:               # m backwards
    loss.backward()
    dist.send(h.grad, dst=0)
opt.step(); opt.zero_grad()
```

- Stage 0 mirrors it: send each `h`, then `recv` each gradient and call `h.backward(g)`.
- Divide the loss by **m**: accumulated gradient = batch mean.
- Same order on both stages → no deadlock.

Measuring the bubble

```
busy = 0.0
def timed(fn, *args):
    global busy
    t = time.perf_counter()
    out = fn(*args)       # sync first on GPU
    busy += time.perf_counter() - t
    return out

t0 = time.perf_counter()
run_step(timed)                  # one GPipe step
wall = time.perf_counter() - t0
print(1 - busy / wall)           # measured bubble
```

$$\text{measured} = 1 - \text{busy}/\text{wall}$$

$$\text{predicted} = (p-1)/(m+p-1)$$

Watch out

Expect measured $>$ predicted: per-microbatch **overheads** do not shrink with b.

Exercise 1: solutions

m	1	2	4	8
predicted bubble	50%	33%	20%	11%
message per microbatch	16 KB	8 KB	4 KB	2 KB

Watch out

Messages of a few KB are **latency-bound**; each microbatch also pays launch overhead. For our tiny MLP, a deeper pipeline does **not** pay off.

In our pipeline

Lab 5: report predicted vs measured bubble and **explain the gap**.

Exercise 2: cost four 3D layouts

(d, t, p)	m	GB / GPU	Tensor	Bubble	Data	Step
A. (8, 8, 1)						
B. (4, 16, 1)						
C. (4, 8, 2)						
D. (2, 8, 4)						

GIVEN

$\Psi = 40$B, 48 layers, h $= 8192$, s $= 2048$; 64 sequences per step, b $= 1$; compute 0.99 s

8 GPUs per server; NVLink 450 GB/s, InfiniBand 50 GB/s per direction

Which layouts fit in 80 GB? Which one is fastest?

Source: H100 and DGX H100 datasheets (link speeds); cost model of Part 3.

Exercise 2: solutions

(d, t, p)	m	GB / GPU	Tensor	Bubble	Data	Step
A. (8, 8, 1)	8	$83 > 80$	0.20 s	0.0%	0.35 s	—
B. (4, 16, 1)	16	42	3.87 s	0.0%	0.15 s	5.01 s
C. (4, 8, 2)	16	43	0.20 s	6.2%	0.15 s	1.44 s
D. (2, 8, 4)	32	23	0.20 s	9.4%	0.05 s	1.40 s

Key idea

A does not fit; B puts tensor traffic on InfiniBand (**3.5× slower**). C and D are within 3%; the model favours **D**.

Source: Computed with the cost model of Part 3; Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.

Justifying the choice

- Both fit; both keep tensor traffic on **NVLink**.
- D: bubble and sends cost $+0.06$ s; data traffic on **InfiniBand** saves 0.10 s.
- D leaves **57 GB** free: room for a larger b or less recomputation.
- C and D differ by $< \mathbf{3\%}$: smaller than the model's uncertainty.

In our pipeline

Use the model to **eliminate** layouts (A, B) and to **rank** the rest; measure the top two.

Key takeaways

- Pipeline parallelism cuts the model **by layers**; only activations ($b \cdot s \cdot h$) cross a cut.
- Naive schedule: one GPU busy at a time. Microbatches give bubble $\mathbf{(p-1)/(m+p-1)}$.
- **1F1B** keeps the bubble and bounds activations in flight to $\mathbf{p}$ (GPipe: m).
- Layout: $\mathbf{N = d \cdot t \cdot p}$; t inside a server, p across servers, d outermost.
- Memory $\approx \mathbf{16\Psi/(t \cdot p)}$ + activations; make $t \cdot p$ just big enough to fit.
- A simple cost model eliminates bad layouts; **measure** the close ones.

Next time, and readings

When	What
After this lecture	T3 checkpoint: correct DP update and basic layout tests
After this lecture	Lab 5 due: tensor and pipeline partitions, bubbles, 3D layout
Lecture 2 (this week)	Diagnosing: profiles, stragglers, MFU, energy, checkpoints

READ

Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021 (§2–3).

Huang et al., "GPipe," NeurIPS 2019.